

Invert Lab

DOTS Sprite Animator

User Guide

Version 0.8.1

Package: com.invertlab.spriteanimator

Namespace: InvertLab.Sprites.DOTS

Unity 6000.0+ · Entities · Entities Graphics · URP

Portable DOTS-first 2D flipbook animation: timeline authoring, events, colliders, sockets, combat playback helpers, CPU/GPU paths, and optional Unity Physics.

License: Unity Asset Store EULA (see LICENSE.md).

README.md

DOTS Sprite Animator

Portable Unity 6000.0 package for DOTS-first 2D flipbook animation authoring and runtime playback.

Current package version: **0.8.1**. Full guide: `Documentation~/Documentation.md` / `Documentation~/DOTS-Sprite-Animator-User-Guide-v0.8.0.pdf`.

Published by **Invert Lab**.

License: see `LICENSE.md` (Unity Asset Store EULA).

Mental model

See `Documentation~/Architecture.md` (Profile -> Play -> optional colliders) and `Documentation~/Samples.md`.

Scope

- Publisher: **Invert Lab**.
- Package id: `com.invertlab.spriteanimator` (internal; not shown on the store).
- Runtime namespace: `InvertLab.Sprites.DOTS`.
- Runtime assembly: `InvertLab.SpriteAnimator.Runtime`.
- No gameplay-framework dependencies (no NZCore, Rukhanka, Trove, or project-only assemblies).
- Supported dependencies: Entities, Entities Graphics, and URP.

Samples

Optional — not imported until you choose them in Package Manager (**Samples** tab). See `Documentation~/Samples.md`.

Open the tools

- Window > DOTS Sprite Animator
- Tools > DOTS Sprite Animator > Validate Installation
- Tools > DOTS Sprite Animator > Help

Toolbar highlights

- **New Profile / Load Profile** / **Save Profile**
- Step-frame transport |< < > >|
- Play/Pause, Stop, Loop, playback speed
- Undo/Redo text buttons (Windows-safe)

- Help and install validation shortcuts

Timeline and preview

- Continuous playhead mapping for Loop, Once, Ping Pong, and Reverse Loop.
- Ruler scrubbing and drag-driven frame reorder/resize/events.
- Preview zoom + pan.
- Per-frame TRS keys:
 - Position offset
 - Scale
 - Rotation
 - Tween mode (Linear, SmoothStep, EaseIn, EaseOut, Step)

GPU eligibility badge

Each clip shows either:

- **GPU clock OK** (uniform sequential loop/once clip), or
- **CPU only** (uses offsets, holds, events, reorder, ping-pong/reverse, TRS tween, sockets, etc.).

`SpriteGpuAnimSwitch.ToGpu` keeps refusing non-eligible clips.

Colliders and sockets

- Author square/circle/polygon colliders per frame.
- Copy colliders to next frame or all frames.
- Click **Add Socket**, then click the preview to place a named attach point (`index:Name`). The same name is one identity across frames; position (px) and angle (deg) can differ per frame. Sockets stay CPU-only.

Facing groups

Clips can be grouped under a logical facing group (for 4-way / 8-way sets) with explicit facing direction metadata.

Runtime integration

1. Add `SpriteAnimSetAuthoring` and assign a profile or direct clip data.

2. Playback:

- `SpriteAnims.Play(entityManager, entity, "Run")`
- `SpriteAnims.PlayFacing(entityManager, entity, "Walk", SpriteFacingDirection.DownLeft)`

See:

- `Documentation~/QuickStart.md`

- `Documentation~/AnimationEvents.md`

QuickStart.md

DOTS Sprite Animator Quick Start

Publisher: **Invert Lab**. Package version **0.8.1**.

1) Install

Embed or reference the package:

- Packages/com.invertlab.spriteanimator

Required packages:

- com.unity.entities
- com.unity.entities.graphics
- com.unity.render-pipelines.universal

Use **Tools > DOTS Sprite Animator > Validate Installation** after import.

2) Create a profile

1. Open **Window > DOTS Sprite Animator**.
2. Click **New Profile**.
3. Assign your spritesheet texture, rows, and columns.
4. Add or load clips (accordion UI; edit FPS, wrap, interrupt, priority).
5. Click **Save Profile**.

The profile saves as:

- <SheetName>_profile.asset
- <SheetName>_profile.json

3) Author animation data

Per clip:

- Wrap mode (Loop / Once / Ping Pong / Reverse Loop / **ReverseOnce**)
- Interrupt (Always / Never / AfterTime) + Priority + optional OnCompleteClipIndex
- Frame order + hold durations + FPS
- Exact-time frame events (EVENT TYPES; multi-select delete; marker RMB menu)
- Optional facing group + direction metadata
- Combo window fields for combat follow-ups

Per frame:

- Sheet column
- Position offset, Scale, Rotation, TRS tween
- Colliders (Square / Circle / Polygon; Frame / Clip / Character; Unity bake/gizmos)
- Sockets: **Add Socket**, click preview; inventories; Independent Motion layers

Inspector: **Show Sprite** toggles scene MeshRenderer preview (ShowSpriteInScene).

4) Runtime setup

1. Add SpriteAnimSetAuthoring to a GameObject.
2. Assign Profile.
3. Bake to entities (SubScene or conversion flow).
4. Optional: SpriteAnimPlayerAuthoring for managed Play / Hitstop / queue APIs.

Runtime calls:

```
`csharp
SpriteAnims.Play(entityManager, entity, "Run");
SpriteAnims.PlayFacing(entityManager, entity, "Walk", SpriteFacingDirection.Down);
player.PlayOneShot("Attack");
player.Hitstop(0.12f);
`
```

5) GPU vs CPU path

The **GPU clock** is a simple flipbook path: the shader picks the frame from a global clock. It is **not** a full feature substitute for the CPU player.

Clip / inspector eligibility:

- **GPU clock OK**: simple uniform sequential **Loop** / **Once** clips
- **CPU only**: events, custom holds, reorder, offsets, sockets, TRS tween, ping-pong / reverse / ReverseOnce

SpriteGpuAnimSwitch.ToGpu / SpriteAnims.TryToGpu only accept eligible clips.

Playback Path (SpriteAnimSetAuthoring)

Mode	Behavior
Auto	CPU default (same as before).
PreferGpu	After bake, convert once when eligible. Promotes a single uniform sheet onto the legacy GPU setSheet path.
ForceCpu	Stay on (or return to) the CPU clock.

PreferGpu stays on CPU when:

- the clip is not GPU-eligible
- the profile uses **multiple sheets**
- the sheet uses **Cropped** cell layout (needs CPU CropST)

6) Samples

Samples are **optional**. Import from Package Manager → DOTS Sprite Animator → Samples.

See Samples.md. Do not expect Assets/Samples until you import.

Architecture

See **Architecture.md** for the Profile -> Play -> optional collider diagram.

See **Samples.md** for which sample to open for each feature.

Optional: Unity Physics

Install com.unity.physics only if you need Method = UnityPhysics. Core animation and Query/Unity2D do not require it.

Architecture.md

Architecture (mental model)

Publisher: **Invert Lab**. Package: `com.invertlab.spriteanimator`.

Core animation pipeline

```
Profile (Window > DOTS Sprite Animator)
-> clips / frames / TRS / events / sockets / collider boxes
SpriteAnimSetAuthoring (+ optional SpriteAnimPlayerAuthoring)
-> bake (SubScene) or hybrid preview
Play (CPU player, or GPU for simple flipbooks)
-> SpriteInstance / GPU render
```

Animation works with **no** collider method and **no** Unity Physics package.

Optional collider layers

Method	What it does	Needs
Query	Frame/Clip/Character boxes stay data; SpriteHitboxQuery AABB	nothing extra
Unity2D	Spawns Collider2D children	built-in 2D physics
UnityPhysics	Character body -> Unity Physics Box/Sphere/Convex; frame stays AABB query	optional <code>com.unity.physics</code>

Lifetime rules (Unity Physics)

- **Character** — persistent hurtbox. Bake Colliders (edit) or `SpriteUnityPhysicsHurtbox.Ensure` (Play / open scene).
- **Frame / Clip** — not baked into Physics. Read AABB via `SpriteHitboxQuery`, then optionally `overlapAabb` against Character bodies.

```
Attack frame AABB --OverlapAabb--> Character PhysicsCollider (hurtbox)
```

Optional Unity Physics assembly

- Assembly: `InvertLab.SpriteAnimator.UnityPhysics`
- Compiles only when `com.unity.physics` is installed (`INVERTLAB_UNITY_PHYSICS`).
- Core `InvertLab.SpriteAnimator.Runtime` never hard-depends on Physics.

Two Unity Physics workflows

1. **Bake now** — Method = `UnityPhysics` → **Bake Colliders** → `UnityPhysicsColliders` children (`Box/Sphere/Convex` preview).
2. **Skip bake** — on Play, call `SpriteUnityPhysicsHurtbox.Ensure(transform, animSet)` (open-scene sprites are not `SubScene`-baked).

PlaybackPath / GPU flipbook limits

`SpriteAnimSetAuthoring.PlaybackPath`:

- **Auto** — CPU default.
- **PreferGpu** — convert once after spawn when the clip is GPU-eligible.
- **ForceCpu** — never use the GPU clock.

The GPU path is a **simple flipbook** (shader clock on one legacy sheet):

- Single sheet only (multi-sheet characters stay on CPU).
- Uniform grid cells (Cropped / per-cell CropST stays on CPU).
- No sockets, animation events, frame reorder, custom holds, or per-frame TRS.

Use `PreferGpu` for dense crowds / simple loops. Keep heroes and VFX on CPU when you need events or sockets.

API: `SpriteAnims.TryToGpu / ToCpu / IsGpuDriven`.

See also

- `QuickStart.md`
- `Samples.md`
- `PureOverlap/PureOverlap.md`

Samples

Optional `Samples~` — import via Package Manager Samples tab. See `Samples.md`.

Samples.md

Samples map

Samples ship inside the package as **optional** `Samples~` folders. They are **not** downloaded into your project until you import them.

How to import

1. Window → Package Manager
2. Select **DOTS Sprite Animator** (In Project / My Registries / embedded)
3. Open the **Samples** tab
4. Import only what you need

Imported path (Unity default):

```
`Assets/Samples/DOTS Sprite Animator/<version>/<Sample display name>/`
```

What each sample proves

Sample (Package Manager name)	Proves	Needs
Playback API	Play / one-shot / facing	—
Events	Frame events	—
Sockets	Socket attach points	—
Crowd GPU	GPU / crowd scale	—
Collider Query	Query + AABB hits	—
Collider Unity 2D Events	Unity 2D collider children	—
Unity Physics	Character Physics body + frame AABB OverlapAabb	com.unity.physics
Showcase (Clembod)	Art/profiles demo	credit Clembod

Unity Physics note

Scripts live in the **Unity Physics** sample; related scenes may be under **Collider Unity 2D Events** until fully split. Open-scene actors use `SpriteUnityPhysicsHurtbox.Ensure` (not SubScene bake).

Documentation.md

Invert Lab DOTS Sprite Animator — User Guide

Product: Invert Lab DOTS Sprite Animator

Package id: com.invertlab.spriteanimator

Version: 0.8.1

Publisher: Invert Lab

Namespace: InvertLab.Sprites.DOTS

Unity: 6000.0+ (Entities, Entities Graphics, URP)

This guide covers the shipped 0.8.1 feature set: playback control, combat helpers, wrap modes, editor UX, sockets, events, crowd/GPU overview, and samples.

1. Install and open tools

Embed or reference:

- Packages/com.invertlab.spriteanimator

Required packages:

- com.unity.entities
- com.unity.entities.graphics
- com.unity.render-pipelines.universal

Menus:

Menu	Purpose
Window > DOTS Sprite Animator	Main timeline / sheet authoring window
Tools > DOTS Sprite Animator > Open Window	Same window
Tools > DOTS Sprite Animator > Validate Installation	Dependency check
Tools > DOTS Sprite Animator > Help	Opens QuickStart / README
Tools > DOTS Sprite Animator > Build Sockets Sample	Rebuild sockets sample
Tools > DOTS Sprite Animator > Build Events Sample	Rebuild events sample
Tools > DOTS Sprite Animator > Build Playback API Sample	Rebuild playback API sample
Tools > DOTS Sprite Animator > Open Crowd GPU Sample	Open crowd/GPU sample scene
Tools > DOTS Sprite Animator > Setup Authoring Example Scene	Authoring scene helper

After import, run **Validate Installation**.

2. Quick authoring workflow

1. Open **Window > DOTS Sprite Animator**.
2. **New Profile**, assign spritesheet texture, rows, columns.
3. Add / edit clips (accordion UI), frames, FPS, events, colliders, sockets.
4. **Save Profile** → <SheetName>_profile.asset + .json.
5. On a GameObject: add SpriteAnimSetAuthoring, assign Profile, bake (SubScene / conversion).
6. Optional: SpriteAnimPlayerAuthoring for managed playback helpers (Play, Hitstop, queue, etc.).

Runtime basics:

```
`csharp
using InvertLab.Sprites.DOTS;

SpriteAnims.Play(entityManager, entity, "Run");
SpriteAnims.PlayFacing(entityManager, entity, "Walk", SpriteFacingDirection.Down);
`
```

3. Playback API (0.8)

APIs exist on both SpriteAnims (ECS) and SpriteAnimPlayerAuthoring (managed). Prefer orce: true only when you intentionally bypass interrupt / priority gates.

3.1 Interrupt and Priority

Per clip:

Field	Meaning
Interrupt Always	Any Play may replace this clip (idle / walk).
Interrupt Never	Locked until Once/ReverseOnce completes, or Stop / orce.
Interrupt AfterTime	Cancelable only when normalized time $\geq$ CancelAfter (0–1).
Priority	Higher (or equal) priority may interrupt the current clip when orce is false.

Combo windows temporarily lower the *effective* current priority via ComboWindowPriorityBoost, so follow-up attacks can land during the window.

3.2 Queue / One-shot / chaining

API	Behavior
PlayOrQueue(...)	Tries Play; if blocked by interrupt/priority and queueIfBlocked, stores a queued clip that plays when the current clip finishes.
PlayOneShot(...)	Plays a clip once, then resumes the previous locomotion/idle clip.
OnCompleteClipIndex	When a Once/ReverseOnce clip finishes, automatically starts this clip index (-1 = none).

3.3 Speed, seek, freeze, blend

API	Behavior
SetSpeed / GetSpeed	Playback rate. Negative rewinds; 0 freezes the clock.
Pause / Resume	Pause without losing speed restore intent.
Freeze / Unfreeze	Aliases of Pause / Resume.
SeekFrame / SeekNormalized / SetTime	Jump within the current clip.
Blend	Crossfade weight 1 → 0 during fade (gameplay / shader readable; no dual draw).

3.4 Lifecycle callbacks

Callback	When
SpriteAnimEvents.ClipStarted / authoring ClipStarted	Clip begins (also reserved event Ids 250).
SpriteAnimEvents.ClipCompleted / authoring ClipCompleted	Once-style clip ends (Id 251). Completes at phase 0 when finishing with negative speed.

Managed:

```
`csharp
player.ClipStarted += clipIndex => { / ... / };
player.ClipCompleted += clipIndex => { / ... / };
`
```

ECS / static:

```
`csharp
SpriteAnimEvents.ClipStarted += (entity, clipIndex) => { };
SpriteAnimEvents.ClipCompleted += (entity, clipIndex) => { };
SpriteAnimEvents.Raised += (entity, evt) => { / frame markers / };
`
```

4. Combat helpers

API	Behavior
Hitstop(seconds)	Freeze the clock for a duration, then restore previous speed. Shares timer with Hold.
Hold(seconds)	Same shared freeze timer.
HoldAtFrame(frame, seconds)	SeekFrame then Hold.
Combo window	Clip fields ComboWindowStartFrame / EndFrame / PriorityBoost. Query with InComboWindow(); TryComboPlay uses Always interrupt + priority boost in-window.
SetFacing / PlayMirrored / Play/PlayFacing flipX overloads	Facing / mirror helpers for 2D combat.
PlayRandomStart	Play clip and seek to a random frame (crowd variety).
PlayWeighted	Weighted random among clip indices (NativeArray, managed arrays, or up to 4 pairs).

Hitstop / Hold use simulation / authoring delta time (HitstopRemaining, HitstopRestoreSpeed, HitstopActive).

5. Wrap modes

Mode	Behavior
Loop	Repeats forward.
Once	Plays forward once, then completes.
Ping Pong	Forward then reverse, repeating.
Reverse Loop	Loops while playing backward.
ReverseOnce (0.8)	Plays once from last frame toward first (positive Speed; not a Speed=-1 hack), then completes. Supports OnCompleteClipIndex / queue drain like Once.

Clips using ping-pong, reverse, custom holds, events, sockets, or TRS tween stay **CPU only**.

6. Editor UX (0.8)

- **Clip accordion** — compact clip list with expand/collapse.
- **FPS edits** — per-clip frame rate authoring.
- **EVENT TYPES** — multi-select and delete for profile event type rows.
- **Event-marker RMB menu** — right-click markers for edit / type / delete actions.
- **Rename UX** — cleaned control IDs; frame-delete persists correctly on Save Profile.

- **Show Sprite** — scene preview toggle (renamed from Show Scene Preview; serialized as ShowSpriteInScene). Draws via MeshRenderer edit preview when on; play mode still uses SpriteInstanceRenderSystem when applicable.
- **Sockets** — named attach points; click-to-place in preview; inventories; Independent Motion layers + triggers.
- **Unity collider bake / gizmos** — square / circle / polygon colliders with Frame / Clip / Character lifetime; Physics Query AABB / Unity 2D / Both; scene gizmos and bake into runtime data.

Toolbar still includes New / Load / Save Profile, step transport [`< > >`], Play/Pause, Loop, speed, Undo/Redo, Help.

7. Sockets, independent motion, inventory

- Socket **Name** is an editor label; **ID** (prefer dotted ids like equipment.head, combat.muzzle) is the stable gameplay contract.
- **Frame-attached** sockets follow the current frame key (position px, angle, scale).
- **Independent Motion** sockets run their own timeline; RMB on the row adds triggers that fire SpriteSocketEventBuffer / SpriteSocketEvents.Raised.
- **Inventory** patterns help organize equipment / VFX attach points across clips.
- SyncUnitySockets() keeps GameObject socket transforms aligned with the current frame (authoring / preview path).
- Attach with SpriteSocketAttachmentAuthoring on a child; baker resolves stable ID.
- Runtime lookup: SpriteSockets.TryGetWorldPose / TryGetPose.
- Sockets force **CPU** playback (not GPU clock).

See also Documentation~/SocketGameplayAPI.md.

8. Animation events (Footstep / Attack story)

Author markers in the timeline window. Each marker has:

- Byte **Id** (profile EVENT TYPES)
- Exact in-frame timestamp
- Fire mode **Loop** or **Once**
- Optional typed payloads (Int/Float vectors, Bool, Color, Text, Asset GUID hash, ...)

Story pattern: put a Footstep marker on walk contact frames and an Attack hit marker on the impact frame. Receivers spawn dust / SFX / hitboxes from SpriteAnimEventBuffer or SpriteAnimEvents.Raised.

EventMarkers is the source of truth (multiple markers per frame). EventIds[] keeps the first marker per frame for older profiles / GPU eligibility.

Clips with any event markers stay on the CPU player.

See Documentation~/AnimationEvents.md for payload / receiver details.

9. Crowd / GPU path overview

Two playback paths:

Path	Use when
CPU (SpriteAnimPlayerSystem)	Events, sockets, custom holds, reorder, TRS tween, ping-pong / reverse / ReverseOnce, combat helpers needing precise markers.
GPU clock	Simple flipbook only: uniform sequential Loop / Once , single sheet, no crops/sockets/events. Inspector badge GPU clock OK .

SpriteGpuAnimSwitch.ToGpu / SpriteAnims.TryToGpu refuse non-eligible clips. Authoring PlaybackPath PreferGpu converts once when eligible; multi-sheet and Cropped layouts stay on CPU.

For crowds:

- SpriteCrowdSpawnerAuthoring (and sample AuthoringCrowdDemo) primes instance scale / material so GPU sprites render at the intended size.
- Use GPU-eligible idle/walk clips for density; keep hero / VFX on CPU when you need events or sockets.
- Sample: import **Crowd GPU** from Package Manager Samples, then open via **Tools > DOTS Sprite Animator > Open Crowd GPU Sample**.

Preview / edit mesh uses SpriteUnlit2DPreview (separate from instanced runtime shaders).

10. Samples

Import from Package Manager → Samples (lands under Assets/Samples/DOTS Sprite Animator/...):

Folder	What it shows	How to open
ColliderEventExample	Frame / clip / character colliders + event-driven reactions	Open scene under ColliderEventExample/Scenes/
EventsExample	Footstep / Attack markers, ClipStarted / ClipCompleted	Tools > DOTS Sprite Animator > Build Events Sample , then open scene
PlaybackApiExample	Play, PlayOneShot, PlayOrQueue, Priority, Hitstop, Hold (keys 1–6)	Tools > ... > Build Playback API Sample
CrowdGpuExample	Dense GPU / crowd spawn path	Tools > ... > Open Crowd GPU Sample
Sockets	Weapon socket attach, Idle/Attack, CPU path	Tools > ... > Build Sockets Sample ; open SocketsExample.unity
Showcase/Clembod	Warrior / Bringer Of Death art for demos	See License.txt / Contact.txt — credit Clembod appreciated

Showcase art © Clembod (personal/commercial use OK; do not redistribute/resell the art alone). Credit not required but appreciated: [[@Clembod](https://clembod.itch.io)](https://clembod.itch.io).

11. Runtime integration checklist

1. Profile authored and saved.
2. SpriteAnimSetAuthoring on entity root with Profile assigned.
3. Optional SpriteAnimPlayerAuthoring for MonoBehaviour-driven gameplay.
4. For ECS receivers: systems [UpdateAfter(typeof(SpriteAnimPlayerSystem))] reading SpriteAnimEventBuffer with SpriteAnimEventsPending.
5. Choose CPU vs GPU per clip eligibility; do not force GPU on event/socket clips.
6. Validate Installation before packaging builds.

12. Related docs in this package

- Documentation~/QuickStart.md — short install path
- Documentation~/AnimationEvents.md — markers, payloads, receivers
- Documentation~/SocketGameplayAPI.md — socket IDs, attachments, independent triggers
- CHANGELOG.md — version history
- README.md — package overview

© Invert Lab. DOTS Sprite Animator 0.8.1.

LICENSE.md

License

DOTS Sprite Animator

Copyright (c) 2026 Invert Lab

This package is licensed for use under the **Unity Asset Store End User License Agreement (EULA)**.

- Full terms: <https://unity.com/legal/as-terms>
- Purchasing or downloading this asset from the Unity Asset Store constitutes acceptance of that EULA.
- You may use this asset in your Unity projects as allowed by the Asset Store EULA.
- You may not redistribute this package (or a substantial portion of it) as a competing Asset Store / package product except as the EULA expressly allows.

Showcase sample art may include third-party content with its own credit notes inside that sample. Those notes do not replace this package license.

Publisher: Invert Lab

Package id: `com.invertlab.spriteanimator`

CHANGELOG.md

Changelog

[0.8.1] - 2026-09-08

- Samples moved into optional UPM Samples~ (Package Manager Samples tab). Project no longer ships a baked Assets/Samples copy.
- Optional Unity Physics assembly `InvertLab.SpriteAnimator.UnityPhysics` (`INVERTLAB_UNITY_PHYSICS`). Core Runtime no longer hard-references `com.unity.physics`.
- Authoring `PlaybackPath`: `Auto` / `PreferGpu` / `ForceCpu`. `PreferGpu` promotes a single uniform sheet to the legacy GPU clock when eligible; multi-sheet and Cropped layouts stay on CPU.
- Public helpers: `SpriteAnims.TryToGpu` / `ToCpu` / `IsGpuDriven`.
- Install validation reports optional `com.unity.physics` as info (not required).
- Package `LICENSE.md` points at the Unity Asset Store EULA.
- Docs: GPU path honesty (simple flipbook limits) + `PlaybackPath` notes.

[0.8.0] - 2026-09-02

- Scene sockets: convert pivot-relative pixel poses to bottom-center mesh local so independent/frame sockets (e.g. `HealthBarSocket`) match the animator preview after the cell pivot change. Formula: $\text{meshLocal} = ((\text{pivot} - (0.5, 0)) * \text{cellSize} + \text{pixels}) / \text{PPU}$.
- Added Bake Pivot (default on) — empty Pivot child at local (0,0) — and Show Scene Pivot gizmo (crosshair + label).
- Editor: renamed inspector toggle **Show Scene Preview** -> **Show Sprite** (`ShowSpriteInScene`, Formerly `SerializedAs`).
- Combat/playback pack: `Hitstop/Hold/HoldAtFrame` (shared `HitstopRemaining/HitstopRestoreSpeed/HitstopActive` timer; simulation delta), combo window (`ComboWindowStartFrame/EndFrame/PriorityBoost` on clip + blob; `InComboWindow/TryComboPlay`; interrupt Always + priority boost in window), facing helpers (`SetFacing`, `PlayMirrored`, `Play/PlayFacing` flipX overloads), `PlayRandomStart`, `PlayWeighted` (`NativeArray` + managed + up to 4 pairs).
- Clip extras: `SetSpeed/GetSpeed` (negative = rewind, 0 = freeze clock), `Pause/Resume/Freeze/Unfreeze`, `SeekFrame/SeekNormalized/SetTime` on `SpriteAnims` and `SpriteAnimatorAuthoring`.
- Lifecycle: `SpriteAnimEvents.ClipStarted` / `ClipCompleted` (+ reserved Ids 250/251). Once + negative speed completes at phase 0.
- Playback control: clip Interrupt modes `Always` / `Never` / `AfterTime`, plus clip Priority gating for Play.
- Queue helpers: `PlayOrQueue` and `PlayOneShot` on the authoring player.
- Clip `OnCompleteClipIndex` chaining when a once-shot finishes.
- Crossfade: `Blend weight` (1->0) exposed for gameplay / shaders during fade.
- Sockets: `SyncUnitySockets` keeps `GameObject` socket transforms aligned with the current frame.
- Editor: rename UX control IDs cleaned up; frame-delete now persists correctly on Save Profile.
- Package cleanup: removed broken `BallForge/legacy` Examples scene; control focus names use `InvertLab` prefixes.

0.7.0 - 2026-08-27

- Rebranded package ownership under **InvertLab**:
 - Package id: `com.invertlab.spriteanimator`
 - Namespace: `InvertLab.Sprites.DOTS`
 - Assemblies: `InvertLab.SpriteAnimator.*`
- Renamed package display branding to **DOTS Sprite Animator** and updated menu paths to:
 - Window > DOTS Sprite Animator
 - Tools > DOTS Sprite Animator > *
- Added per-frame sockets (named local attach points) to clip data and runtime blob.
- Click-to-place sockets in the preview (Add Socket → balloon → click frame). Same name is one identity across frames; `LocalPosition` and `LocalAngle` are per-frame keys copied into `SpriteSocketBuffer` (CPU playback, no socket tween).
- Added collider copy workflows:
 - Copy current frame colliders to next frame
 - Copy current frame colliders to all frames
- Added facing metadata for 4-way / 8-way clip grouping (`Facing Group + Facing`).
- Added preview zoom + pan controls.
- Added `Documentation~/QuickStart.md` and updated package docs for 0.7 workflows.

0.6.0 - 2026-08-27

- Added toolbar profile actions: **New Profile** and **Load Profile...**
- Added per-frame TRS authoring channels and blob/runtime support:
 - Position offset
 - Scale
 - Rotation
 - Tween mode
- Added built-in easing helper (`Linear`, `SmoothStep`, `EaseIn`, `EaseOut`, `Step`).
- Added GPU eligibility checks and inspector badge:
 - **GPU clock OK** for simple sequential Loop/Once clips
 - **CPU only** for clips using advanced channels (offsets, holds, events, reorder, ping-pong/reverse, TRS tween, sockets)
- Extended `SpriteGpuAnimSwitch.ToGpu` gating to keep refusing non-eligible clips.
- Added UV flip support on both instanced and GPU-anim shader paths (no 180° transform hack).
- Added step-frame transport controls (`|<`, `<`, `>`, `>|`) and left/right shortcuts when no onion ghost is selected.

0.5.1 - 2026-08-27

- Fixed timeline playhead mapping so the red needle is a continuous function of preview time across frame boundaries and wrap modes (Loop, Once, Ping Pong, Reverse Loop).
- Kept pointer-follow behavior only during active scrub/reorder/resize/event drags.
- Defaulted preview playback to paused on open.
- Disabled Play when no clip is available.
- Updated toolbar Undo/Redo labels to plain text for Windows compatibility.
- Disabled Save Profile until a sheet is assigned.
- Added installation validator dialog feedback on both success and failure (with Console logs).
- Added Help entry points:
 - Tools > DOTS Sprite Animator > Help

(Truncated in PDF — see CHANGELOG.md in package.)